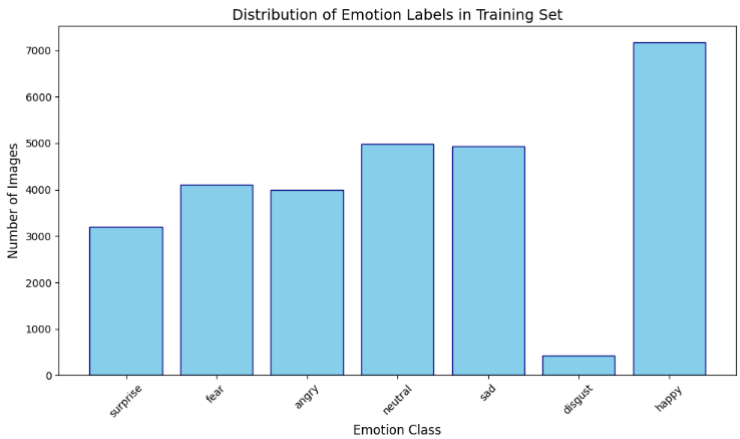
CG3201 Machine Learning and Deep Learning
Lab 3 – Convolution Neural Network

0. Preliminary

1) We will use Pytorch extensively from now. This is a useful resource to learn Pytorch: https://www.learnpytorch.io/
2) Locate "Put your code here comments to fill in the template"
3) Please read through the code to understand the pipeline. We already put some comments there to explain the code. If you don't understand anything, feel free to ask the TA.

1. Task Introduction and Dataset

In this lab, we will explore how to predict emotion from human facial expression image. Go to this Kaggle dataset, upload the template notebook and grab your free GPU with a cost of queuing 😊.
We have 7 categories: angry, disgust, fear, happy, neutral, sad, surprise. Each image is a grayscale image and is of size (48, 48).
This is the distribution of the categories from the training set:



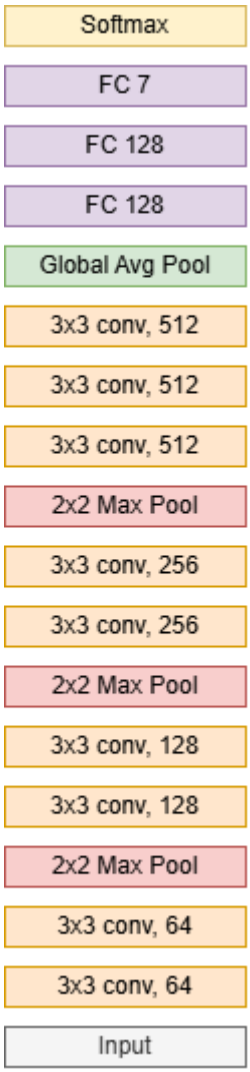
As you can see the dataset is not balanced where we have a lot of happy images and only a few disgust images. We will perform weighted sampling later to make sure we don't overfit on one category while we can still learn the under-represented category.

2. Data Preprocessing and Loading

We split the training dataset into training and validation with ratio of 0.8:0.2. We also apply data augmentation on the training data to help the model learn different variations. As we want to balance the dataset during training, we will apply WeightedRandomSampler, meaning class with more samples have lower weight and class with less samples have higher weight. Classes with higher weight will be sampled more frequently during training. Moreover, training, validation and test data will go through the same normalization step. Lastly, we put the dataset into dataloader which yields 128 images for each loading step.

3. Model Architecture

This is our model architecture:



Note that all the convolution layers have padding of 1, stride of 1 and max pooling layers have stride of 2. Meanwhile, global average pooling layer takes the average over the entire feature map of a given channel. Hence, it will reduce the number of parameters drastically in the first fully connected layer. To stabilize the training and add regularization, we will use batch norm for convolution layers and dropout with dropout rate of 20% for the first 2 fully connected layers with 128 neurons.

4. Training

1) Optimizer: We will Use Adam optimizer from Pytorch with learning rate of 0.001 and use default values for other hyperparameters of the optimizer.
2) Loss function: We will use Cross Entropy loss.
3) Validation strategy: We will evaluate the validation data using f1-score after every epoch and save the model checkpoint with the highest f1-score for evaluation on test data later.

4) We will train over 80 epochs. If you don't have time, just interrupt the cell and make sure the checkpoint for best epoch is saved. Each epoch should take roughly 12 - 15s.

5. Testing

Load checkpoint with the highest f1-score and make predictions on test data. You need to compute the accuracy, f1-score, confusion matrix of the predictions.

END